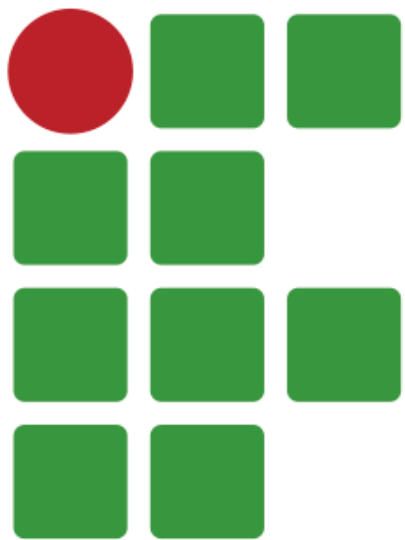




**INSTITUTO
FEDERAL**
Brasília

Algoritmos e Programação de Computadores TAI-35

Prof. MSc. Thiago Neves
IFB Campus Taguatinga
25/06/2026

Tópicos Avançados

Ponteiros em C

Ponteiros

- ❑ Computadores modernos possuem memórias grandes e rápidas. Você sabe que o tamanho da memória é expresso em unidades chamadas bytes, e também sabe que, ao declarar qualquer variável, essa variável ocupa uma pequena parte da memória do computador.
- ❑ Até agora, tudo o que precisávamos saber era qual valor estava armazenado na variável. A partir de agora, também estamos interessados em saber onde esse valor está armazenado.
- ❑ Tente perceber esta diferença importante:
 - o **valor** da variável é aquilo que a variável armazena;
 - o **endereço** da variável é a informação sobre onde essa variável está localizada, ou seja, onde ela “vive”.

Ponteiros

Ponteiros são usados para armazenar informações sobre a localização, isto é, o endereço, de outros dados. Podemos dizer que ponteiros são como placas de sinalização. Eles não dizem nada sobre o lugar em si, mas mostram claramente como chegar até ele.

Declaração de ponteiros em C

Em C, um **ponteiro** é uma variável que armazena o **endereço de memória** de outra variável.

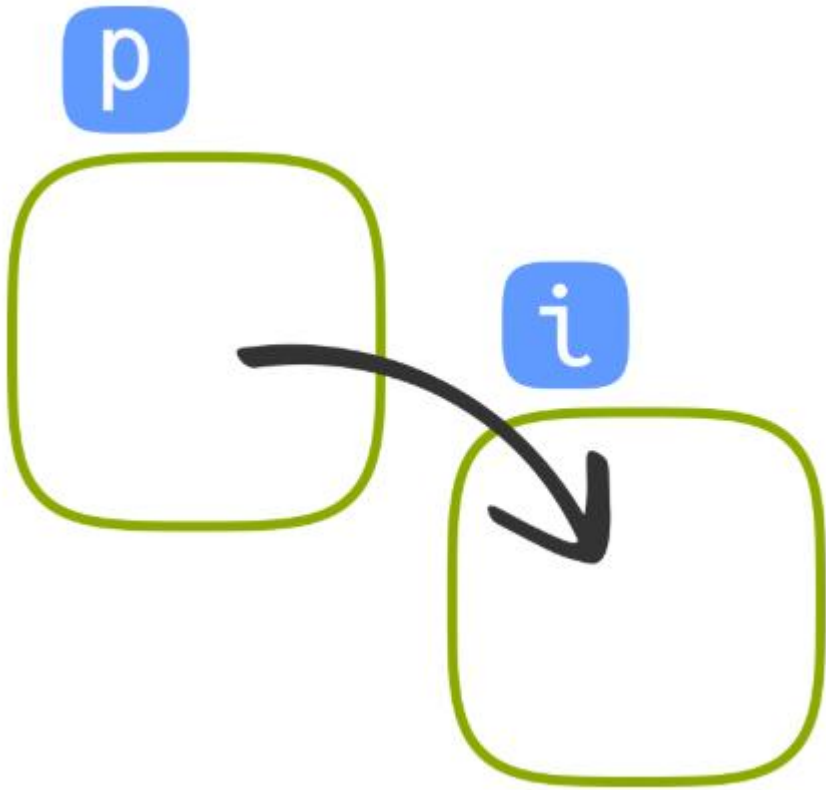
```
int *a;      // declara um ponteiro de inteiro "a".
```

```
*a = 10;     // ao acessar o endereço de memória "a", passa a  
             // existir o valor "10"
```

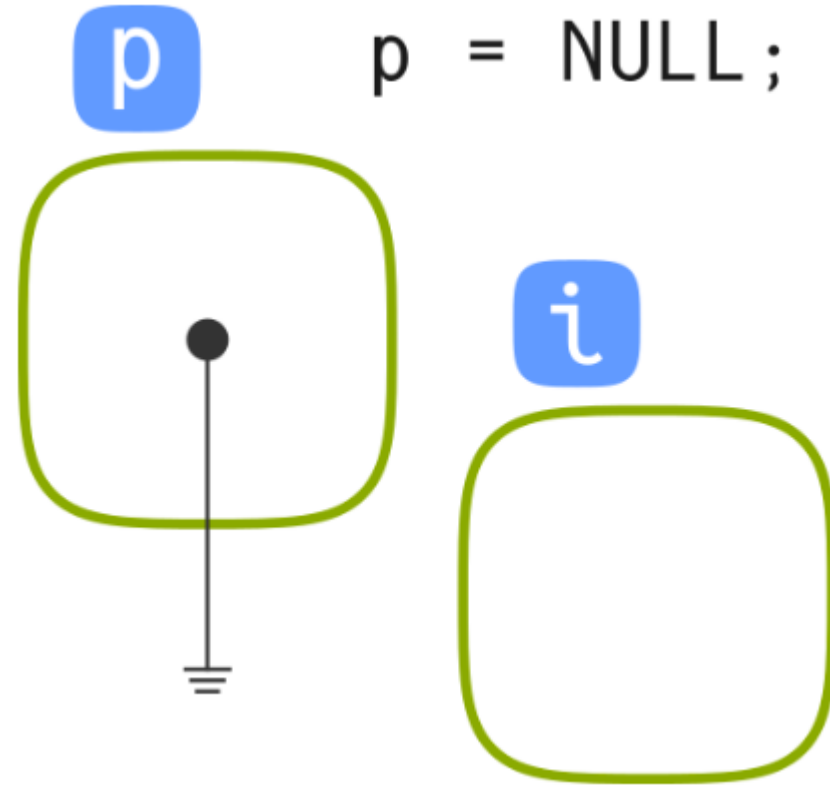
```
Int *b = a;  // "b" recebe o endereço de memória de "a".
```

```
A = NULL;   // macro que é igual a 0, usada para iniciar ponteiros.
```

p = &i;



p = NULL;



Operador sizeof

```
#include <stdio.h>

int main(void) {
    printf("This computing environment uses:\n");
    printf("%zu byte for chars\n", sizeof(char));
    printf("%zu bytes for shorts\n", sizeof(short int));
    printf("%zu bytes for ints\n", sizeof(int));
    printf("%zu bytes for longs\n", sizeof(long int));
    printf("%zu bytes for long longs\n", sizeof(long long int));
    printf("%zu bytes for floats\n", sizeof(float));
    printf("%zu bytes for doubles\n", sizeof(double));
    printf("%zu bytes for pointers\n", sizeof(int *));
    return 0;
}
```

// %zu serve como especificador de formato na linguagem de programação C, projetado especificamente para imprimir variáveis do tipo `size_t`.

// `size_t` é um tipo inteiro sem sinal usado para representar o tamanho de objetos em bytes. Ele é comumente retornado pelo operador `sizeof` em C ao determinar o tamanho de um tipo de dado ou de uma variável.

Tabela de prioridades em C

1	!, ~ (type), ++, --, +, -, *, &, sizeof
2	*, /, %
3	+, -
4	<<, >>
5	<, <=, >, >=
6	==, !=
7	&
8	
9	&&
10	
11	=, +=, -=, *=, /=, %=, &=, ^=, =, >>=, <<=

Exemplo 1: Declaração de ponteiros

```
#include <stdio.h>
```

```
int main() {  
    int idade = 30;  
    int *ptr = &idade;  
  
    printf("Valor de idade: %d\n", idade);  
    printf("Endereco de idade: %p\n", &idade);  
    printf("Valor armazenado em ptr: %p\n", ptr);  
    printf("Valor apontado por ptr: %d\n", *ptr);  
  
    return 0;  
}
```

Inicialização de ponteiros

Errado (ptr não aponta para uma área válida de memória)

```
int *ptr;  
*ptr = 10;
```

Correto (Agora ptr aponta para x)

```
int x;  
int *ptr = &x;
```

Exemplo 2: Alterando uma variável usando ponteiro

```
#include <stdio.h>
```

```
int main() {
```

```
    int numero = 10;
```

```
    int *ptr = &numero;
```

```
    printf("Antes: %d\n", numero);
```

```
    *ptr = 50;
```

```
    printf("Depois: %d\n", numero);
```

```
    return 0;
```

```
}
```

Passagem de variáveis por referência

Em C, é possível passar argumentos por valor (cópia) ou por referência (endereços de memória) através do uso de ponteiros. Ponteiros são passados para as funções como qualquer outra variável (porém, com sintaxe própria).

Exemplo:

```
void swap(int *x, int *y) {  
    int temp;  
    temp = *x;           // salva o valor de x  
    *x = *y;             // faz x receber o valor de y  
    *y = temp;           // faz y receber o valor anterior de x  
}  
  
void main() {  
    int i=10, j=20;  
    swap(&i, &j);  
}
```

Exemplo 3: Declaração de ponteiros e passagem de variáveis por referência

```
#include <stdio.h>

void play(int *nro) {
    *nro += 1;
    printf("play: %d\n", *nro);
}

int main() {
    int nro = 10;
    printf("main: %d\n", nro);
    nro += 1;
    printf("main: %d\n", nro);
    nro += 1;
    printf("main: %d\n", nro);
    nro += 1;
    play(&nro);
    printf("main: %d\n", nro);
    return 0;
}
```

Exemplo 4: Passagem de variáveis por valor VS por referência.

```
#include <stdio.h>

void play1(char a) {
    printf("Valor: <%c> Memoria <%d>\n", a, &a);
}

void play2(char *a) {
    printf("Valor: <%c> Memoria <%d>\n", *a, a);
}

int main() {
    char a = 'A';
    printf("Valor: <%c> Memoria <%d>\n", a, &a);
    play1(a);
    play2(&a);

    return 0;
}
```

Exemplo 5: Passagem de arrays por referência

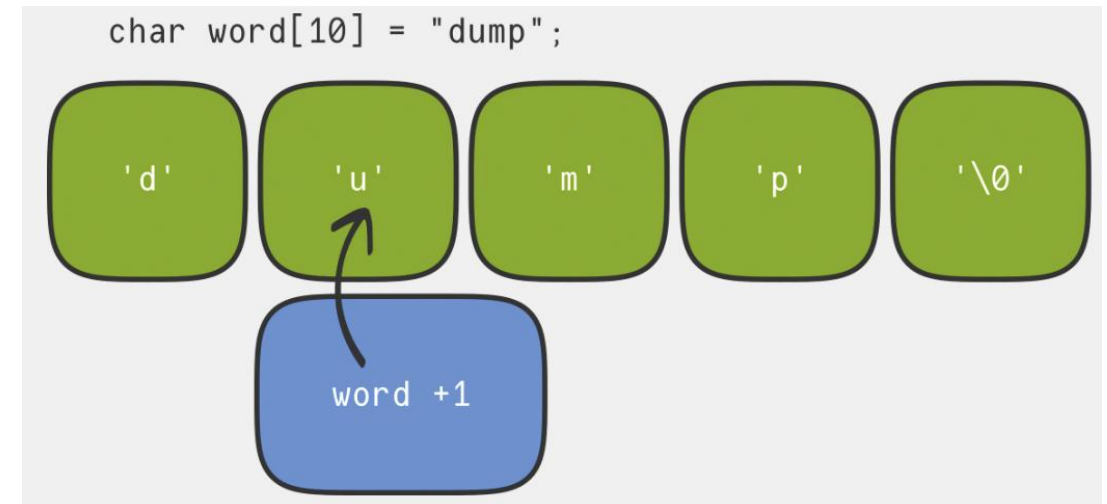
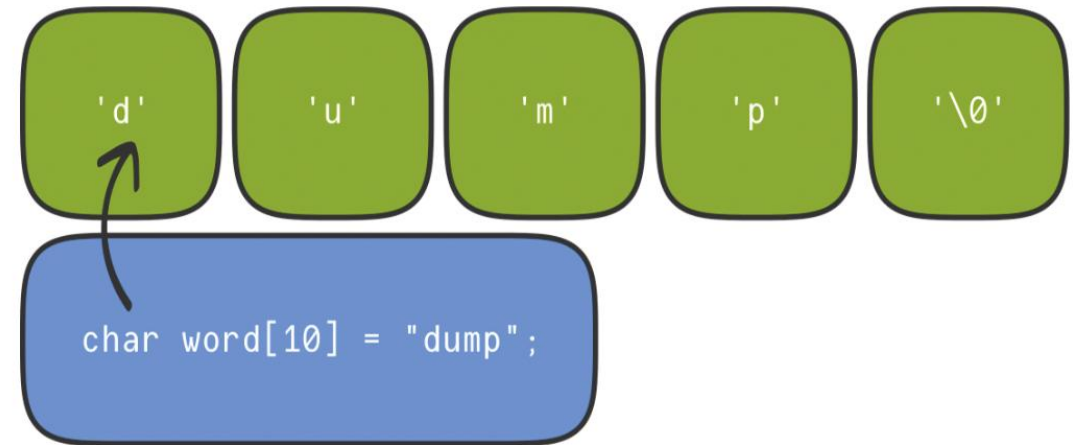
```
#include <stdio.h>

void dobrarElementos(int v[], int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        v[i] = v[i] * 2;
    }
}

int main() {
    int numeros[5] = {1, 2, 3, 4, 5};
    dobrarElementos(numeros, 5);
    printf("Array após a função:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", numeros[i]);
    }
    return 0;
}
```

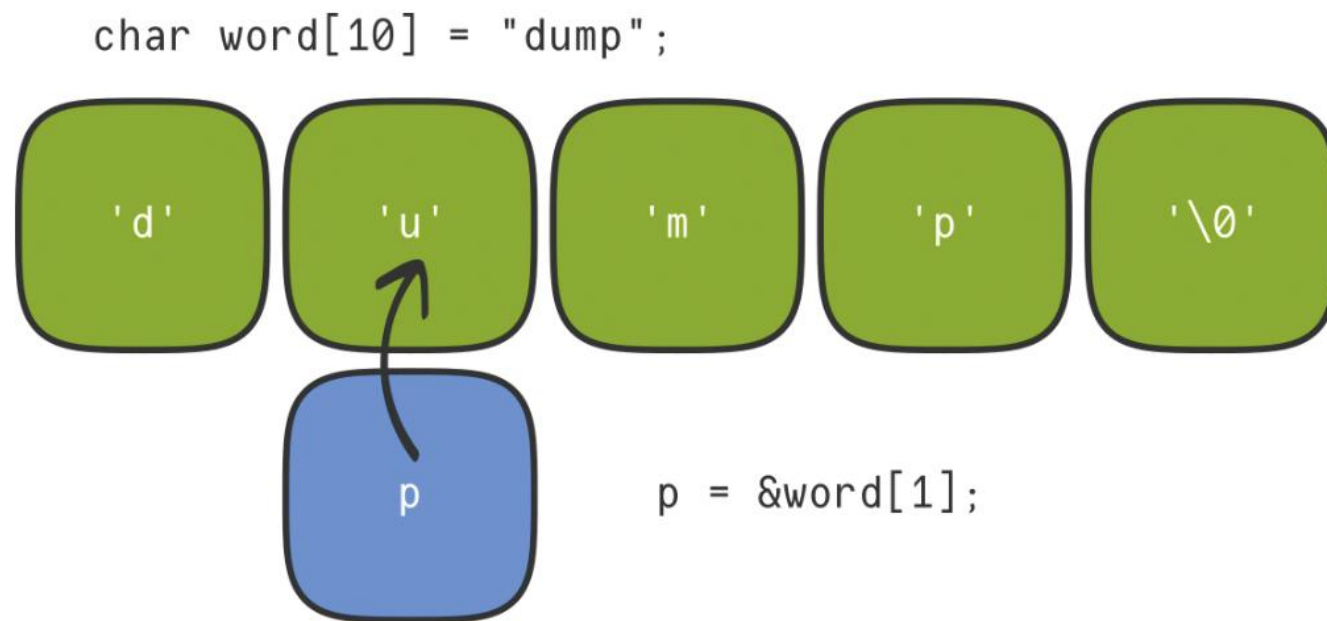
Indexação de Arrays

`*(word + 1) = 'a';`
igual a
`word[1] = 'a';`



Indexação de Arrays

```
char string[] = "ABC";  
char *p;
```



Erros comuns

Erro 1: Usar um ponteiro sem inicialização

Você comete esse erro quando usa um ponteiro que não recebeu previamente nenhum valor útil e que **não aponta para uma variável existente ou para um espaço válido de memória**. O compilador não consegue detectar o erro, pois sua natureza só é revelada em tempo de execução.

```
1
2  #include <stdio.h>
3  #include <string.h>
4
5  int main(void) {
6      char *ptr;
7      strcpy(ptr, "you may get into trouble soon");
8      puts(ptr);
9      return 0;
10 }
11
```

Erro 2: Exceder o tamanho do Array

Isso acontece quando você se esquece do tamanho declarado do seu array.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main(void) {
5      char str[10];
6
7      strcpy(str, "Welcome to Troubleland!");
8      printf("%s", str);
9      return 0;
10 }
```

Erro 3: String como ponteiro

```
char nome[] = "Instituto Federal";  
char *nome = "Instituto Federal";
```

No primeiro caso, temos um array modificável. No segundo caso, o ponteiro aponta para uma string literal, que geralmente não deve ser alterada.

Perigoso!

```
char *nome = " Instituto Federal";  
nome[0] = 'P';
```

Alocação Dinâmica de Memória

Memória sob demanda

- ❑ Nos exemplos que mostramos até agora, o gerenciamento de memória ocorreu sem que percebêssemos. As partes da memória que usamos para armazenar valores ficaram ocultas por trás dos nomes de escalares e arrays.
- ❑ Elas aparecem assim que são declaradas e desaparecem quando nosso programa encerra sua execução. Todo o trabalho associado à alocação de memória foi organizado pelo compilador, e nós não precisamos nos preocupar com o funcionamento disso. É assim que deve ser: linguagens de alto nível e seus compiladores são projetados para livrar a mente dos desenvolvedores dessas tarefas rotineiras.
- ❑ Mas, muitas vezes, o desenvolvedor deseja ter controle total sobre quanta memória é usada e exatamente quando ela é usada. Isso é especialmente importante quando você não sabe, antecipadamente, qual será o tamanho dos dados a serem processados.

Função Malloc

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *v = malloc(5 * sizeof(int));
    if (v == NULL) {
        printf("Erro ao alocar memoria.\n");
        return 1;
    }
    for (int i = 0; i < 5; i++) {
        v[i] = i + 1;
    }
    for (int i = 0; i < 5; i++) {
        printf("%d\n", v[i]);
    }
    free(v);
    return 0;
}
```

// A função malloc aloca uma quantidade de bytes.

// A função free libera o bloco de memória.

Função Calloc

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *v = calloc(5, sizeof(int));    // A função calloc aloca memória e inicializa tudo com zero.

    if (v == NULL) {
        printf("Erro ao alocar memoria.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d\n", v[i]);
    }

    free(v);
    return 0;
}
```

Ponteiros e structs

```
#include <stdio.h>

struct Aluno {
    char nome[50];
    int idade;
    float media;
};

int main() {
    struct Aluno a = {"Carlos", 20, 8.5};
    struct Aluno *ptr = &a;

    printf("Nome: %s\n", ptr->nome);
    printf("Idade: %d\n", ptr->idade);
    printf("Media: %.2f\n", ptr->media);

    return 0;
}
```

Alocando structs dinamicamente

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Aluno {
    char nome[50]; int idade; float media;
};

int main() {
    struct Aluno *a = malloc(sizeof(struct Aluno));
    if (a == NULL) {
        printf("Erro ao alocar memoria.\n");
        return 1;
    }
    strcpy(a->nome, "Mariana");
    a->idade = 22;
    a->media = 9.3;
    printf("Nome: %s\n", a->nome);
    printf("Idade: %d\n", a->idade);
    printf("Media: %.2f\n", a->media);
    free(a);
    return 0;
}
```

Alocação dinâmica de matriz

```
#include <stdio.h>
#include <stdlib.h>

int **criarMatriz(int linhas, int colunas) {
    int **matriz = malloc(linhas * sizeof(int *));
    if (matriz == NULL) return NULL;
    for (int i = 0; i < linhas; i++) {
        matriz[i] = calloc(colunas, sizeof(int));
        if (matriz[i] == NULL) {
            for (int j = 0; j < i; j++) free(matriz[j]);
            free(matriz);
            return NULL;
        }
    }
    return matriz;
}

void liberarMatriz(int **matriz, int linhas) {
    if (matriz == NULL) return;
    for (int i = 0; i < linhas; i++) free(matriz[i]);
    free(matriz);
}
```

Alocação dinâmica de matriz

```
int main() {  
    int linhas = 3;  
    int colunas = 4;  
    int **matriz = criarMatriz(linhas, colunas);  
  
    if (matriz == NULL) {  
        printf("Erro ao alocar matriz.\n");  
        return 1;  
    }  
    matriz[0][0] = 10;  
    matriz[1][2] = 20;  
    matriz[2][3] = 30;  
    for (int i = 0; i < linhas; i++) {  
        for (int j = 0; j < colunas; j++) printf("%d\t", matriz[i][j]);  
  
        printf("\n");  
    }  
    liberarMatriz(matriz, linhas);  
    return 0;  
}
```

Manipulação de string usando ponteiros

```
#include <stdio.h>

void inverterString(char *str) {
    char *inicio = str;
    char *fim = str;
    char temp;
    while (*fim != '\0') fim++;
    fim--;
    while (inicio < fim) {
        temp = *inicio;
        *inicio = *fim;
        *fim = temp;
        inicio++;
        fim--;
    }
}

int main() {
    char texto[] = "ponteiro";
    inverterString(texto);
    printf("%s\n", texto);
    return 0;
}
```

Exercício 1

Crie um vetor de 10 inteiros e imprima todos os valores usando apenas aritmética de ponteiros.

Não use: `v[i]`

Use: `*(v + i)`

Exercício 2

Crie uma função que receba um vetor e seu tamanho. A função deve calcular a soma dos elementos usando ponteiros.

Protótipo: `int somar(int *v, int tamanho);`

Exercício 3

Crie um vetor dinâmico de inteiros usando malloc. O usuário deve informar o tamanho do vetor.

Depois, o programa deve:

- Ler os valores;
- Imprimir os valores;
- Calcular a média;
- Liberar a memória.

Exercício 4

Crie uma estrutura chamada Aluno com:

```
char nome[50];  
int idade;  
float media;
```

Aloque dinamicamente um aluno, leia os dados, imprima os dados e libere a memória.

Exercício 5

Crie uma calculadora usando ponteiros para função.

O programa deve ter funções para:

- somar
- subtrair
- multiplicar
- Dividir

E deve usar um ponteiro para função para chamar a operação escolhida pelo usuário.

Exercício 6 (Avançado)

Crie um programa em C que implemente uma calculadora usando ponteiros para função.

O programa deve possuir as funções:

```
int somar(int a, int b);  
int subtrair(int a, int b);  
int multiplicar(int a, int b);  
int dividir(int a, int b);
```

Depois, crie uma função chamada:

```
int executarOperacao(int a, int b, int (*operacao)(int, int));
```

Essa função deve receber dois inteiros e um ponteiro para função, executando a operação indicada.

O programa principal deve permitir ao usuário escolher a operação desejada e chamar executarOperacao.

A divisão por zero deve ser tratada adequadamente.

Exercício 7 (Avançado)

Implemente manualmente a função:

```
void inverterString(char *str);
```

A função deve inverter a string recebida **sem usar arrays auxiliares e sem usar funções da biblioteca string.h**.

Exemplo:

```
char texto[] = "ponteiro";  
inverterString(texto);  
printf("%s", texto);
```

Saída esperada:

orietnop

A solução deve usar apenas ponteiros para percorrer e modificar a string.

Ponteiros para funções

Ponteiros para funções

Um ponteiro para função é uma variável que armazena o endereço de uma função. Assim como um ponteiro para um dado comum (como int ou char), um ponteiro para função pode ser usado para chamar a função indiretamente, passar funções como argumento, ou armazená-las em estruturas.

- ❑ **Callbacks:** Passar uma função como argumento para outra.
- ❑ **Flexibilidade:** Escolher qual função chamar em tempo de execução.
- ❑ **Organização:** Criar tabelas de funções ou listas dinâmicas.
- ❑ **Plugins:** Implementar comportamentos variáveis.

Declaração

**retorno (*nome_do_ponteiro)(tipo_parametro1,
tipo_parametro2, ...);**

- ❑ **retorno**: tipo de dado que a função retorna (ex: int, void etc.).
- ❑ ***nome_do_ponteiro**: o nome do ponteiro para a função.
- ❑ **(tipo_parametro1, tipo_parametro2, ...)**: os tipos de parâmetros que a função aceita.

Exemplo Básico

```
#include <stdio.h>
int somar(int a, int b) {
    return a + b;
}
int multiplicar(int a, int b) {
    return a * b;
}
int main() {
    int (*operacao)(int, int);
    operacao = somar;
    printf("%d\n", operacao(2, 3));
    operacao = multiplicar;
    printf("%d\n", operacao(2, 3));
    return 0;
}
```

Ponteiros como argumentos

```
#include <stdio.h>

// Duas funções simples
int somar(int a, int b) {
    return a + b;
}

int subtrair(int a, int b) {
    return a - b;
}

// Função que recebe um ponteiro para função
int calcular(int x, int y, int (*operacao)(int, int)) {
    return operacao(x, y);
}

int main() {
    printf("Soma: %d\n", calcular(5, 3, somar));
    printf("Subtração: %d\n", calcular(5, 3, subtrair));

    return 0;
}
```

Usando o typedef para simplificação

```
#include <stdio.h>

typedef int (*Operacao)(int, int);

int somar(int a, int b) { return a + b; }
int subtrair(int a, int b) { return a - b; }

int calcular(int x, int y, Operacao operacao) {
    return operacao(x, y);
}

int main() {
    printf("Soma: %d\n", calcular(5, 3, somar));
    printf("Subtração: %d\n", calcular(5, 3, subtrair));
    return 0;
}
```

Tabela de funções

```
#include <stdio.h>
```

```
int somar(int a, int b) { return a + b; }
```

```
int subtrair(int a, int b) { return a - b; }
```

```
int multiplicar(int a, int b) { return a * b; }
```

```
int main() {
```

```
    // Array de ponteiros para funções
```

```
    int (*operacoes[])(int, int) = {
```

```
        somar,
```

```
        subtrair,
```

```
        multiplicar
```

```
    };
```

```
    // Chamando funções dinamicamente
```

```
    printf("Soma: %d\n", operacoes[0](5, 3));
```

```
    printf("Subtração: %d\n", operacoes[1](5, 3));
```

```
    printf("Multiplicação: %d\n", operacoes[2](5, 3));
```

```
    return 0;
```

```
}
```

Exercício 1

Crie um programa que define duas funções: soma e subtracao. Em seguida, use um ponteiro para função para chamar cada uma delas dinamicamente.

Requisitos:

1. Defina as funções `int soma(int a, int b)` e `int subtracao(int a, int b)`.
2. Crie um ponteiro para função que aceite duas variáveis int e retorne um int.
3. Solicite ao usuário escolher a operação (soma ou subtração).
4. Use o ponteiro para chamar a função correspondente e imprimir o resultado.

Exercício 2

Crie um programa que armazene diferentes operações matemáticas (soma, subtração, multiplicação e divisão) em um array de ponteiros para funções.

Requisitos:

1. Defina as quatro funções básicas de operação matemática.
2. Armazene os ponteiros para essas funções em um array.
3. Solicite ao usuário os dois números e a operação desejada.
4. Utilize o índice do array para chamar a operação e imprimir o resultado.

Exercício 3

Refaça o **Exercício 2**, mas desta vez utilizando typedef para simplificar a declaração do ponteiro para função.

Requisitos:

1. Crie um typedef chamado Operacao para o tipo de ponteiro de função.
2. Armazene os ponteiros em um array.
3. Solicite ao usuário os números e a operação e exiba o resultado.

Operações bitwise em C

Operações bitwise em C

Operações bitwise são aquelas que atuam bit a bit, diretamente sobre os bits dos operandos.

Operador	Nome	Descrição
&	AND	Retorna 1 se ambos os bits forem 1
	OR	Retorna 1 se um dos bits for 1
^	XOR (OU exclusivo)	Retorna 1 se os bits forem diferentes
~	NOT (complemento)	Inverte todos os bits
<<	Shift Left	Desloca os bits para a esquerda (multiplica por 2^n)
>>	Shift Right	Desloca os bits para a direita (divide por 2^n)

Aplicações comuns

- **Verificar se um bit está ativado**

`if (x & (1 << n)) { } // o bit 'n' está ativo`

- **Marcar um bit**

`x = x | (1 << n); // ou: x |= (1 << n);`

- **Desmarcar um bit**

`x = x & ~(1 << n); // ou: x &= ~(1 << n);`

- **Inverter um bit**

`x = x ^ (1 << n); // ou: x ^= (1 << n);`

Exemplo 1

```
#include <stdio.h>
```

```
int main() {  
    unsigned char a = 12; // 00001100  
    unsigned char b = 10; // 00001010  
  
    printf("a & b = %d\n", a & b); // 00001000 = 8  
    printf("a | b = %d\n", a | b); // 00001110 = 14  
    printf("a ^ b = %d\n", a ^ b); // 00000110 = 6  
    printf("~a = %d\n", ~a);      // 11110011 = 243 (em unsigned) ou -13 (em signed)  
    printf("a << 1 = %d\n", a << 1); // 00011000 = 24  
    printf("a >> 1 = %d\n", a >> 1); // 00000110 = 6  
  
    return 0;  
}
```

Exemplo 2

```
#include <stdio.h>
```

```
void print_bits(unsigned char val) {  
    for (int i = 7; i >= 0; i--) {  
        printf("%d", (val >> i) & 1);  
    }  
    printf("\n");  
}
```

```
int main() {  
    unsigned char x = 0b00001111;  
    print_bits(x);  
    // Ativar o 7º bit  
    x |= (1 << 7);  
    print_bits(x); // deve mostrar 10001111  
  
    // Desativar o 3º bit  
    x &= ~(1 << 3);  
    print_bits(x); // deve mostrar 10000111  
  
    // Alternar bit 0  
    x ^= (1 << 0);  
    print_bits(x); // deve mostrar 10000110  
  
    return 0;  
}
```

Exercício 1

Escreva um programa que leia dois números inteiros a e b e exiba:

- $a \& b$
- $a | b$
- $a \wedge b$
- $\sim a$

Mostre o número original e o resultado em binário.

Exercício 2

Leia um número inteiro positivo e um valor n , e exiba:

- O número deslocado para a esquerda ($x \ll n$)
- O número deslocado para a direita ($x \gg n$)

Exercício 3

Implemente três funções (Ativa, Desativa, Alterna):

- `unsigned int set_bit(unsigned int x, int n);`
- `unsigned int clear_bit(unsigned int x, int n);`
- `unsigned int toggle_bit(unsigned int x, int n);`

Depois, leia um número e uma posição de bit e teste as três funções.

Exercício 4

Escreva uma função que conte quantos bits 1 existem em um número inteiro positivo.

Exercício 5

Crie uma função que armazena dois números de 4 bits (0 a 15) em um único unsigned char. Depois, recupere os dois números originais.

- unsigned char compactar(unsigned char a, unsigned char b);
- void descompactar(unsigned char compactado, unsigned char* a, unsigned char* b);